

Déploiement de Nextcloud avec Docker Compose

Mise en place d'un cloud privé conteneurisé

Thibaut Fontaine

Table des matières

1. Introduction	3
1.1. Objectifs du TP	3
1.2. Prérequis	3
1.3. Architecture cible	3
2. Préparation de l'environnement	4
2.1. Création de l'arborescence	4
2.2. Vérification de Docker	4
2.3. Création du réseau Docker	4
3. Configuration Docker Compose	5
3.1. Fichier docker-compose.yml	5
3.2. Fichier d'environnement	6
3.3. Configuration Traefik	7
4. Déploiement	8
4.1. Lancement des conteneurs	8
4.2. Vérification des logs	8
4.3. Accès à l'interface	8
5. Configuration avancée	9
5.1. Optimisation des performances	9
5.1.1. Configuration PHP	9
5.1.2. Configuration du cache	9
5.2. Tâches planifiées (Cron)	9
5.3. Sauvegarde	10
5.3.1. Script de sauvegarde	10
6. Vérification et tests	12
6.1. Tests fonctionnels	12
6.2. Vérification de la santé	12
6.3. Scan de sécurité Nextcloud	12
7. Dépannage	13
7.1. Problèmes courants	13
7.1.1. Erreur de permissions	13
7.1.2. Base de données inaccessible	13
7.1.3. Certificat SSL invalide	13
7.2. Commandes utiles	13
8. Conclusion	15
8.1. Points clés à retenir	15
8.2. Pour aller plus loin	15

1. Introduction

1.1. Objectifs du TP

Ce TP a pour objectif de déployer une instance Nextcloud fonctionnelle en utilisant Docker Compose. Nextcloud est une solution de cloud privé open source permettant le stockage et le partage de fichiers.

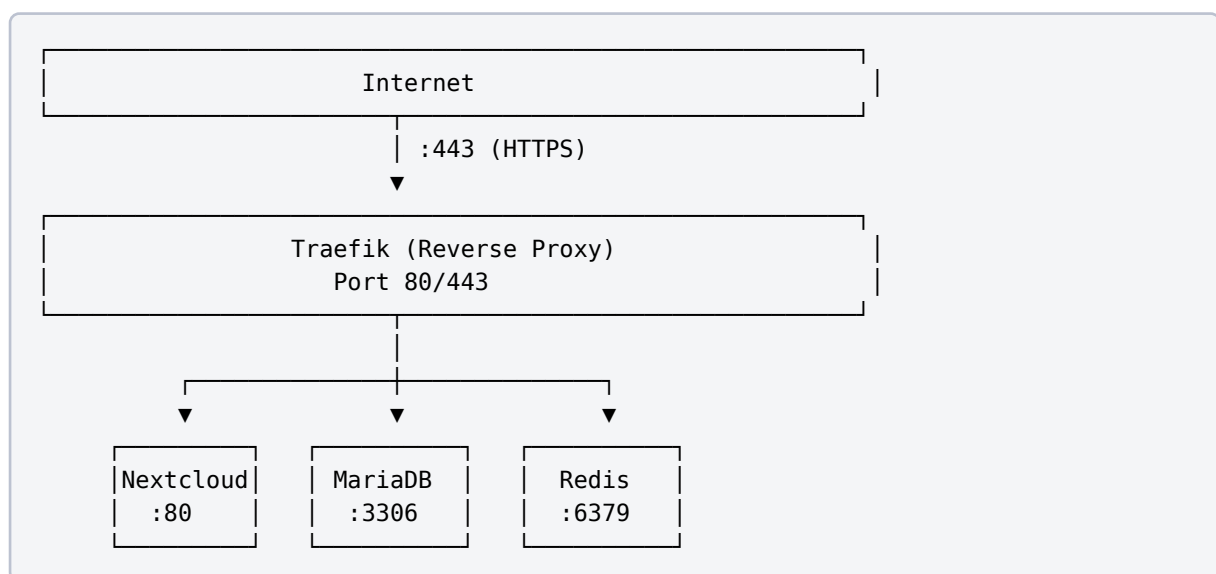
À l'issue de ce TP, vous serez capable de :

- Comprendre l'architecture d'une application multi-conteneurs
- Rédiger un fichier `docker-compose.yml`
- Configurer les volumes persistants
- Mettre en place un reverse proxy avec Traefik
- Sécuriser l'accès avec HTTPS

1.2. Prérequis

- Docker et Docker Compose installés
- Connaissances de base en Linux
- Un nom de domaine (ou utilisation de `localhost`)

1.3. Architecture cible



2. Préparation de l'environnement

2.1. Création de l'arborescence

Créez la structure de répertoires pour le projet :

```
mkdir -p ~/nextcloud-docker/{data,db,redis}  
cd ~/nextcloud-docker
```

2.2. Vérification de Docker

Vérifiez que Docker est correctement installé :

```
docker --version  
# Docker version 24.0.7, build afdd53b  
  
docker compose version  
# Docker Compose version v2.23.0
```

2.3. Création du réseau Docker

Créez un réseau dédié pour l'application :

```
docker network create nextcloud-network
```

3. Configuration Docker Compose

3.1. Fichier docker-compose.yml

Créez le fichier `docker-compose.yml` :

```
version: '3.8'

services:
  # Base de données MariaDB
  db:
    image: mariadb:10.11
    container_name: nextcloud-db
    restart: unless-stopped
    command: --transaction-isolation=READ-COMMITTED --binlog-format=ROW
    volumes:
      - ./db:/var/lib/mysql
    environment:
      - MYSQL_ROOT_PASSWORD=${DB_ROOT_PASSWORD}
      - MYSQL_DATABASE=nextcloud
      - MYSQL_USER=nextcloud
      - MYSQL_PASSWORD=${DB_PASSWORD}
    networks:
      - nextcloud-network

  # Cache Redis
  redis:
    image: redis:7-alpine
    container_name: nextcloud-redis
    restart: unless-stopped
    command: redis-server --requirepass ${REDIS_PASSWORD}
    volumes:
      - ./redis:/data
    networks:
      - nextcloud-network

  # Application Nextcloud
  app:
    image: nextcloud:28-apache
    container_name: nextcloud-app
    restart: unless-stopped
    depends_on:
      - db
      - redis
    volumes:
      - ./data:/var/www/html
    environment:
      - MYSQL_HOST=db
```

```
- MYSQL_DATABASE=nextcloud
- MYSQL_USER=nextcloud
- MYSQL_PASSWORD=${DB_PASSWORD}
- REDIS_HOST=redis
- REDIS_HOST_PASSWORD=${REDIS_PASSWORD}
- NEXTCLOUD_ADMIN_USER=${ADMIN_USER}
- NEXTCLOUD_ADMIN_PASSWORD=${ADMIN_PASSWORD}
- NEXTCLOUD_TRUSTED_DOMAINS=${DOMAIN}
- OVERWRITEPROTOCOL=https

networks:
  - nextcloud-network
labels:
  - "traefik.enable=true"
  - "traefik.http.routers.nextcloud.rule=Host(`${DOMAIN}`)"
  - "traefik.http.routers.nextcloud.entrypoints=websecure"
  - "traefik.http.routers.nextcloud.tls.certresolver=letsencrypt"
  - "traefik.http.services.nextcloud.loadbalancer.server.port=80"

# Reverse Proxy Traefik
traefik:
  image: traefik:v2.10
  container_name: nextcloud-traefik
  restart: unless-stopped
  ports:
    - "80:80"
    - "443:443"
  volumes:
    - /var/run/docker.sock:/var/run/docker.sock:ro
    - ./traefik/acme.json:/acme.json
    - ./traefik/traefik.yml:/traefik.yml:ro
  networks:
    - nextcloud-network

networks:
  nextcloud-network:
    external: true
```

3.2. Fichier d'environnement

Créez le fichier `.env` pour les variables sensibles :

```
# .env
DB_ROOT_PASSWORD=ChangeMe974!
DB_PASSWORD=ChangeMe974Nextcloud!
REDIS_PASSWORD=ChangeMe974
ADMIN_USER=admin
ADMIN_PASSWORD=ChangeMe974
DOMAIN=nextcloud.example.com
```

⚠ Sécurité

Ne jamais commiter le fichier `.env` dans un dépôt Git. Ajoutez-le au `.gitignore`.

3.3. Configuration Traefik

Créez le répertoire et le fichier de configuration Traefik :

```
mkdir -p traefik
touch traefik/acme.json
chmod 600 traefik/acme.json
```

Créez `traefik/traefik.yml` :

```
api:
  dashboard: true

entryPoints:
  web:
    address: ":80"
    http:
      redirections:
        entryPoint:
          to: websecure
          scheme: https

  websecure:
    address: ":443"

providers:
  docker:
    endpoint: "unix:///var/run/docker.sock"
    exposedByDefault: false
    network: nextcloud-network

certificatesResolvers:
  letsencrypt:
    acme:
      email: admin@example.com
      storage: /acme.json
      httpChallenge:
        entryPoint: web
```

4. Déploiement

4.1. Lancement des conteneurs

Démarrez l'ensemble des services :

```
docker compose up -d
```

Vérifiez que tous les conteneurs sont en cours d'exécution :

```
docker compose ps
```

Résultat attendu :

NAME	STATUS	PORTS
nextcloud-app	Up	80/tcp
nextcloud-db	Up	3306/tcp
nextcloud-redis	Up	6379/tcp
nextcloud-traefik	Up	0.0.0.0:80->80/tcp, 0.0.0.0:443->443/tcp

4.2. Vérification des logs

Surveillez les logs pour détecter d'éventuelles erreurs :

```
# Logs de tous les services
docker compose logs -f

# Logs d'un service spécifique
docker compose logs -f app
```

4.3. Accès à l'interface

Accédez à Nextcloud via votre navigateur :

- URL : `https://nextcloud.example.com`
- Utilisateur : `admin`
- Mot de passe : celui défini dans `.env`

5. Configuration avancée

5.1. Optimisation des performances

5.1.1. Configuration PHP

Créez un fichier de configuration PHP personnalisé :

```
mkdir -p config
```

Créez `config/php.ini` :

```
; config/php.ini
memory_limit = 512M
upload_max_filesize = 10G
post_max_size = 10G
max_execution_time = 3600
max_input_time = 3600
```

Ajoutez le volume dans `docker-compose.yml` :

```
app:
  volumes:
    - ./data:/var/www/html
    - ./config/php.ini:/usr/local/etc/php/conf.d/custom.ini:ro
```

5.1.2. Configuration du cache

Éditez la configuration Nextcloud pour optimiser le cache :

```
docker exec -u www-data nextcloud-app php occ config:system:set \
  memcache.local --value="\OC\Memcache\APCu"

docker exec -u www-data nextcloud-app php occ config:system:set \
  memcache.distributed --value="\OC\Memcache\Redis"

docker exec -u www-data nextcloud-app php occ config:system:set \
  memcache.locking --value="\OC\Memcache\Redis"
```

5.2. Tâches planifiées (Cron)

Configurez les tâches de fond via cron :

```
docker exec -u www-data nextcloud-app php occ background:cron
```

Ajoutez un service cron dans `docker-compose.yml` :

```
cron:
  image: nextcloud:28-apache
  container_name: nextcloud-cron
  restart: unless-stopped
  depends_on:
    - app
  volumes:
    - ./data:/var/www/html
  entrypoint: /cron.sh
  networks:
    - nextcloud-network
```

5.3. Sauvegarde

5.3.1. Script de sauvegarde

Créez un script `backup.sh` :

```
#!/bin/bash
# backup.sh

BACKUP_DIR="/backup/nextcloud"
DATE=$(date +%Y%m%d_%H%M%S)

# Création du répertoire de backup
mkdir -p $BACKUP_DIR

# Mise en maintenance
docker exec -u www-data nextcloud-app php occ maintenance:mode --on

# Sauvegarde de la base de données
docker exec nextcloud-db mysqldump -u root -p$DB_ROOT_PASSWORD \
  nextcloud > $BACKUP_DIR/db_$DATE.sql

# Sauvegarde des données
tar -czf $BACKUP_DIR/data_$DATE.tar.gz ./data

# Fin de maintenance
docker exec -u www-data nextcloud-app php occ maintenance:mode --off

echo "Sauvegarde terminée : $DATE"
```

Rendez le script exécutable :

```
chmod +x backup.sh
```

6. Vérification et tests

6.1. Tests fonctionnels

Accès HTTPS	Navigateur	Page de connexion
Connexion admin	Login	Dashboard Nextcloud
Upload fichier	Glisser-déposer	Fichier visible
Partage	Bouton partager	Lien généré
Sync client	Client desktop	Synchronisation OK

Tableau 1 – Checklist des tests fonctionnels

6.2. Vérification de la santé

```
# État des conteneurs
docker compose ps

# Utilisation des ressources
docker stats --no-stream

# Espace disque des volumes
du -sh data/ db/ redis/
```

6.3. Scan de sécurité Nextcloud

Utilisez le scan intégré de Nextcloud :

```
docker exec -u www-data nextcloud-app php occ security:scan
```

Vérifiez également via l'interface d'administration :

- Paramètres > Administration > Vue d'ensemble
- Section «Avertissements de sécurité et configuration»

7. Dépannage

7.1. Problèmes courants

7.1.1. Erreur de permissions

```
# Corriger les permissions des données
docker exec nextcloud-app chown -R www-data:www-data /var/www/html
```

7.1.2. Base de données inaccessible

```
# Vérifier la connectivité
docker exec nextcloud-app ping -c 3 db

# Vérifier les logs MariaDB
docker compose logs db
```

7.1.3. Certificat SSL invalide

```
# Vérifier le fichier acme.json
cat traefik/acme.json | jq .

# Régénérer le certificat
rm traefik/acme.json
touch traefik/acme.json
chmod 600 traefik/acme.json
docker compose restart traefik
```

7.2. Commandes utiles

Redémarrer	<code>docker compose restart</code>
Arrêter	<code>docker compose down</code>
Supprimer tout	<code>docker compose down -v</code>
Shell Nextcloud	<code>docker exec -it nextcloud-app bash</code>
Console OCC	<code>docker exec -u www-data nextcloud-app php occ</code>
Logs temps réel	<code>docker compose logs -f</code>

Tableau 2 – Commandes Docker Compose utiles

8. Conclusion

Ce TP nous a permis de déployer une instance Nextcloud complète et sécurisée avec :

- **Base de données MariaDB** pour le stockage des métadonnées
- **Cache Redis** pour l'amélioration des performances
- **Reverse proxy Traefik** avec certificat SSL automatique
- **Volumes persistants** pour la sauvegarde des données

8.1. Points clés à retenir

1. Docker Compose simplifie le déploiement d'applications multi-conteneurs
2. Les variables d'environnement permettent de sécuriser les secrets
3. Traefik gère automatiquement les certificats Let's Encrypt
4. Les sauvegardes régulières sont essentielles en production

8.2. Pour aller plus loin

- Mettre en place la haute disponibilité avec un cluster
- Configurer la réplication de la base de données
- Intégrer un système de monitoring (Prometheus/Grafana)
- Automatiser les sauvegardes avec un cron job

John Doe

john.doe@etudiant.univ-reunion.fr

+262 6 92 XX XX XX

IUT de La Réunion - 40 avenue de Soweto, 97410 Saint-Pierre
